



THE UNIVERSITY OF
MELBOURNE

SWEN90010 - High Integrity Systems Engineering

Checking Formal Specifications

Toby Murray



toby.murray@unimelb.edu.au



DMD 8.17 (Level 8, Doug McDonnell Bldg)



<http://people.eng.unimelb.edu.au/tobym>



@tobycmurray

Alloy Modelling Overview



signatures: declaring sets and relations

facts: axioms that hold over signatures and globally constrain the model to rule out nonsense

predicates: define relations between variables in a model, used to specify operations, state transitions

functions: shorthand for expressions

Finite State Machines

Begin in an **initial state** s_0

States are just collections of variables of different types.

Have a collection of **operations** that each alter the state.

Each operation implements a different state transition.



Modelling FSMs in Alloy

States are collections of (typed) variables: **sig** State { ... }

Possibly constrained using:
fact *blah* { **always all** s : State | ... }

Initial state: **pred** Init[s : State] { ... }

Operations: **pred** Op₁[s : State] { ... }

pred Op_n[s : State] { ... }

Predicates might make use of functions **fun** as shorthands

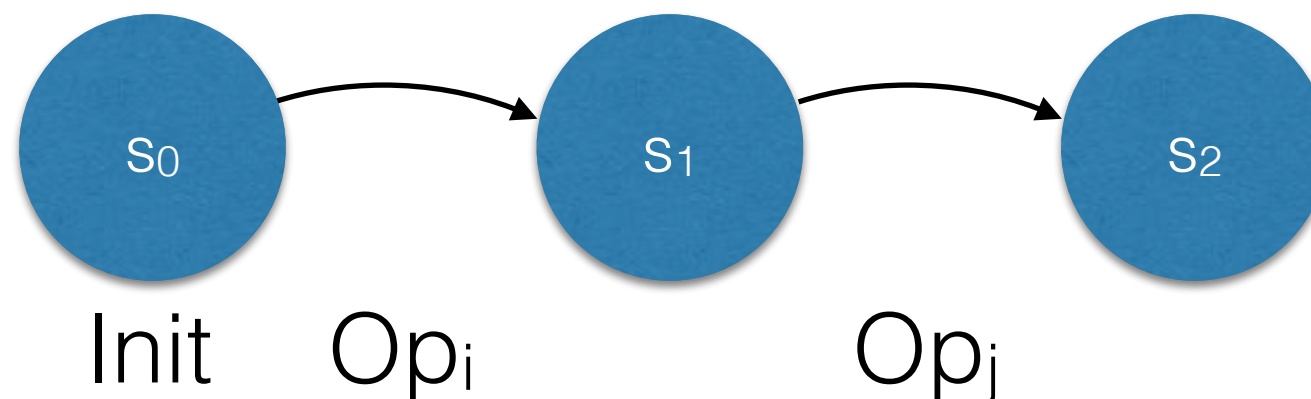
Finite State Machines

Begin in an **initial state** s_0

States are just collections of variables of different types.

Have a collection of **operations** that each alter the state.

Each operation implements a different state transition.



CHECKING SPECIFICATIONS IN ALLOY

“Formal verification is able to tell that a program was built right, but it cannot tell that the right program was built.”

— Gerwin Klein

Checking a Specification



How do we know that our specification
has correctly captured our intentions?

Animation

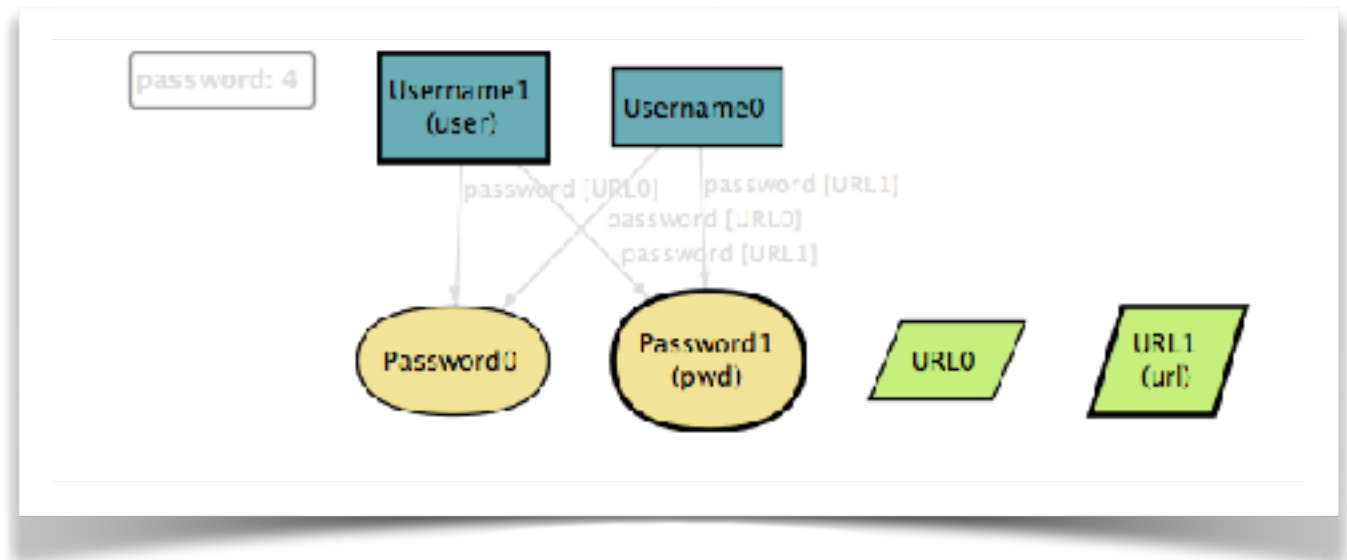
("run" the spec to see what behaviours it captures)

Check Assertions

(see what properties might follow from the spec)

Alloy's **run** command

```
// run the add operation  
run add for 2
```



Finds **instances** that **satisfy** predicates
within a specified **bound**

(begin with smallest reasonable bound,
increase it if unsatisfiable)

Alloy's **assert** statement

```
//If we add a new password, then we get this password when we look it up
assert addWorks {
  all pb : PassBook, url : URL, user : Username, p : Password |
    add [pb, url, user, p] => (after (lookup [pb, url, user]) = p)
}
```

Declare properties that should **follow** from the specification.

Alloy's **check** command

```
check addWorks for 3 but 2 PassBook expect 0
```

Searches up to a fixed bound for **counterexamples** that **violate** a property.

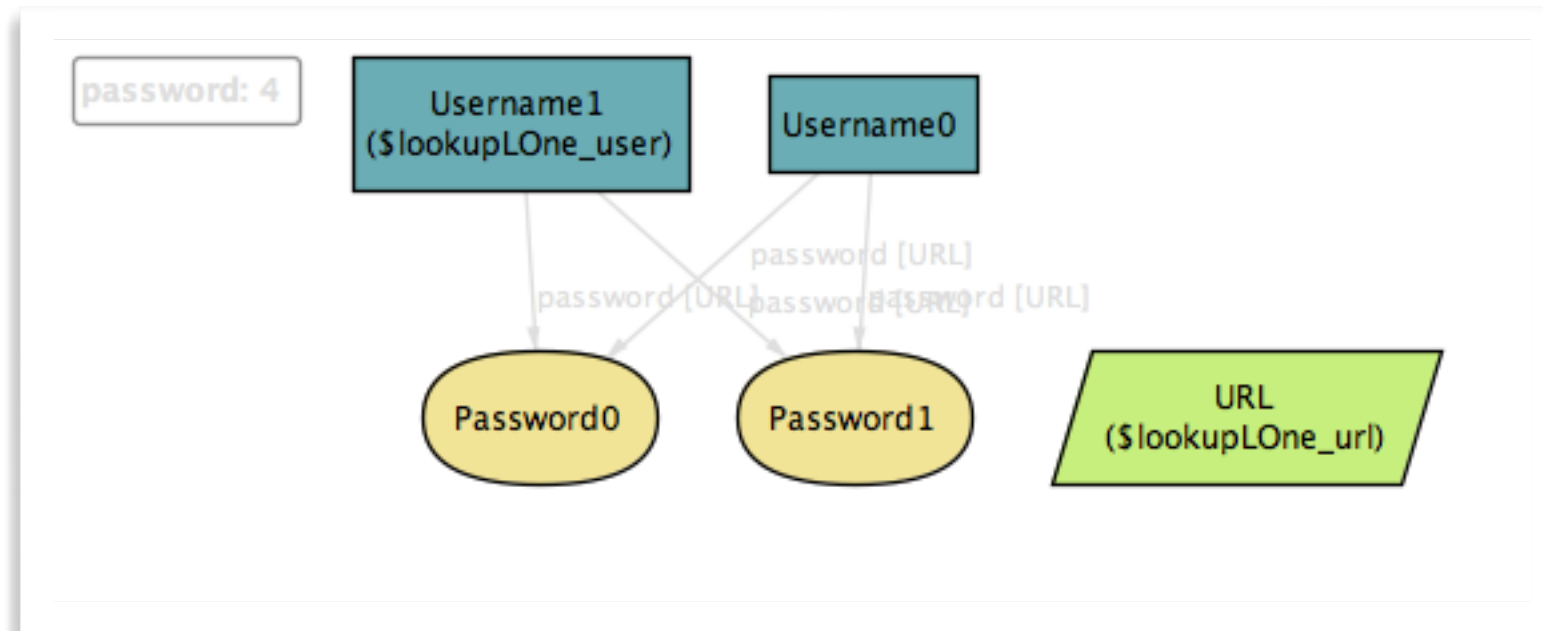
(use “**expect**” to annotate which checks should pass and which should fail)

(begin with smallest reasonable bound, increase it if no counterexample found)

DeletelsUndo Example

```
//If we add and then delete a password, we are back to 'the start'  
assert deletelsUndo {  
  all pb : PassBook, url : URL, user : Username, pwd : Password |  
    (add [pb, url, user, pwd] ; delete [pb, url, user])  
    => pb.password = pb.password"  
}
```

Invalid Counterexamples



Assertions as Tests

Alloy does **bounded** assertion checking

Like testing an assertion for **all** systems up to the bound

Doesn't prove the assertion will hold for systems of arbitrary size in general

Although it might for specific systems and assertions.

In general, it gives **very high confidence** if not a proof.

Assertion Confidence

If Alloy says the assertions all hold, is my spec right?

no

Checks only within finite bounds.

Spec could be incomplete.

Spec could say the wrong thing.